In [5]:
```
# print(12 % 13)
print(-12 % 17)
```

```
5
```

In [6]:
```
l1 = []
l2 = list()
print(type(l1), type(l2))
```

```
<class 'list'> <class 'list'>
```

In [9]:
```
l1.clear()
l1.append(100)
print(l1)
l1.insert(0,200)
print(l1)
```

```
[100]
[200, 100]
```

In [10]:
```
deleted_element = l1.pop()
```

In [12]:
```
print(deleted_element)
print(l1)
```

```
100
[200]
```

In [22]:
```
l1 = ["Sabarish", 10,1200000.0, True]
print(l1)
```

```
['Sabarish', 10, 1200000.0, True]
```

In [23]:
```
del_element = l1.remove(10)
print(del_element)
print(l1)
print(len(l1))
```

```
None
['Sabarish', 1200000.0, True]
3
```

In [24]:
```
a = 10
b = 10
print(id(a), id(b))
```

```
2741942944272 2741942944272
```

In [30]:

```
class Employee:
    def __init__(self):
        self.eid = 0
        self.empName = ""
        self.salary = 0

    def printEmployee(self):
        print("Employee Id : " + str(self.eid))
        print("Employee Name : " + self.empName)
        print("Employee Salary : " + str(self.salary))

emp = Employee()
emp.eid = 1001
emp.empName = "Yuvan"
emp.salary = 1200000

emp1 = Employee()
emp1.eid = 1002
emp1.empName = "Sabarish"
emp1.salary = 1200000


emp.printEmployee()
emp1.printEmployee()
```

```
Employee Id : 1001
Employee Name : Yuvan
Employee Salary : 1200000
Employee Id : 1002
Employee Name : Sabarish
Employee Salary : 1200000
```

In [5]:

```
import sys
class Stack:
    #initializing the stack
    def __init__(self):
        self.elements = []

    #stack operations
    def isEmpty(self):
        #Method 1
#         if len(self.elements)==0:
#             return True
#         else
#             return False
        #method 2
#         return True if len(self.elements) == 0 else return False
#       Method 3
        return len(self.elements) == 0

   #add element into the stack
    def push(self,data):
        self.elements.append(data)

    #remove element from the stack
    def pop(self):
        if self.isEmpty():
            return None
        return self.elements.pop()

    #display top element of stack
    def peek(self):
        if self.isEmpty():
            return None
        return self.elements[-1] # return self.elements[len(elements)-1

    #return the stack size
    def stackSize(self):
        return len(self.elements)

    #display the complete stack details
    def display(self):
        print(self.elements)

# main program
myStack = Stack()
while(True):
    print("Stack Operations Demo")
    print("********************");
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Stack Size")
    print("5. Display the stack")
    print("6. Exit")
    choice = int(input("Enter your choice : "))
    if choice == 1:
        data = int(input("Enter an element : "))
        myStack.push(data)
    elif choice == 2:
        deletedElement = myStack.pop()
        if deletedElement == None:
            print("Stack is Empty...")
```

```
        else:
            print("Removed element : %d"%deletedElement)
    elif choice == 3:
        topElement = myStack.peek()
        if topElement == None:
            print("Stack is empty..")
        else:
            print("Top element : %d"%topElement)
    elif choice == 4:
        print("Stack Size : %d"%myStack.stackSize())
    elif choice == 5:
        myStack.display()
    else:
        print("Thank you so much for being very interactive in the class
#         break
        sys.exit(1)
```

```
Stack Operations Demo
********************
1. Push
2. Pop
3. Peek
4. Stack Size
5. Display the stack
6. Exit
Enter your choice : 6
Thank you so much for being very interactive in the class

An exception has occurred, use %tb to see the full traceback.

SystemExit: 1
```

In [7]:

```
class Node:
    #initialize the node
    def __init__(self, element):
        self.prev = None
        self.value = element
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
    def addFirst(self, element):
        if self.head == None:
            self.head = Node(element)
        else:
            newNode = Node(element)
            newNode.next = self.head
            self.head.prev = newNode
            self.head = newNode
    def addLast(self, element):
        if self.head == None:
            self.head = Node(element)
        else:
            temp = self.head
            newNode = Node(element)
            while(temp.next!=None):
                temp = temp.next
            temp.next = newNode
            newNode.prev = temp
    def printList(self):
        if self.head!=None:
            print ("Forward Traversal ")
            temp = self.head
            while temp.next!=None:
                print(temp.value)
                temp = temp.next
            print(temp.value)
            print("Reverse Traversal ")
            while temp!=None:
                print(temp.value)
                temp = temp.prev

    def deleteFirst(self):
        if self.head != None:
            if self.head.next == None:
                self.head = None
            else:
                self.head = self.head.next
                self.head.prev = None
    def deleteLast(self):
        if self.head != None:
            if self.head.next == None:
                self.head = None
            else:
                temp = self.head
                while temp.next.next != None:
                    temp = temp.next
                temp.next = None


#main functions
myList = LinkedList()
```

```
myList.addFirst(10)
myList.addFirst(20)
myList.addLast(39)
myList.printList()
myList.deleteFirst()
myList.printList()
myList.deleteLast()
myList.printList()
```

```
Forward Traversal
20
10
39
Reverse Traversal
39
10
20
Forward Traversal
10
39
Reverse Traversal
39
10
Forward Traversal
10
Reverse Traversal
10
```

In [ ]: